

claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\workflows\wf_b0e273c7-38a\agent-afc72fdd050d000fa.jsonl`  
- SHA-256: `3690ad3b77c47f892562a3f7647c8ed09671368412fd7c39c6152fe9b46f690e`  
- Source modified: `2026-06-21T08:02:26+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_b0e273c7-38a`  
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`
```

Transcript

1. user (2026-06-21T07:56:51.761Z)

You are reviewing milestone M2 of COMPUTE_DECOUPLED_SERVING in the repo at C:/Users/avidu/Projects/badminton-highlight-indexer.
Spec: docs/COMPUTE_DECOUPLED_SERVING/README.md \$7 row M2 = "input_backend/input_root; wire main CLI/API input through StorageRef.parse_uri (local=identity); storage-fake tests". M2 must be SAFE and
default-OFF: with the default config (storage.input_backend='local', input_root=''), the loaded input
path must be byte-identical to before ? a bare/local path is read in place with NO copy. A bucket/Drive
URI (gs://, s3://, gdrive://) is fetched to a local scratch file before processing.

The change (read all of these + git diff against origin/master):

- backend/config/models.py: StorageConfig gains input_backend + input_root.
- backend/storage/ref.py: resolve_input_uri() (precedence: explicit scheme > anchored local path > input_root join > input_backend prefix > bare-local).
- backend/storage/__init__.py: acquire_local_input() (local=identity, remote=mkdtemp+fetch) + input_is_local().
- backend/main.py: _execute_processing() (API/job path) + the CLI --video-path block both acquire-then-use the local path (keeping the original URI for the DB record + report); /api/process skips the local os.path.exists() gate for a non-local URI.
- tests: tests/test_input_source.py (new) + 3 tests in tests/test_api_endpoints.py.

Reference the worker's already-shipped pattern (backend/worker/__main__.py:91-105 cmd_infer) and the
path-safety helpers (backend/utils/paths.py: validate_input_path / allowed_video_root). Ground EVERY
claim in code with file:line. Give the exact triggering input + wrong-vs-right behaviour. Prefer FEWER,
HIGH-CONFIDENCE findings. A clean bill is a valid, good result.

LENS: byte-identical-local / zero behaviour change. Prove that with the
DEFAULT config a bare/local path flows EXACTLY as before through BOTH the API
(_execute_processing) and
the CLI block. (a) acquire_local_input must return the SAME string object/value for a local path
(no
normalization). Check resolve_input_uri for a normal Windows path "C:\v\x.mp4", a relative
"x.mp4", a
forward-slash "C:/v/x.mp4", and "/data/x.mp4" ? none may be altered when input_root=''. (b) Did
the wiring
miss any file-reading use of req.video_path / args.video_path that still passes the URI (so a
non-local
input would break), OR conversely replace a use that must stay the URI (DB record / report

provenance)?
(c) Confirm compute_video_id now hashes the FETCHED bytes for remote (correct identity) and the same local file for local. (d) The CLI --debug-clip / --trim utility paths still use args.video_path directly ? is that an acceptable scope boundary or a correctness gap?

2. assistant (2026-06-21T07:56:54.194Z)

I'll review milestone M2 of COMPUTE_DECOUPLED_SERVING. Let me start by pulling the repo and examining the changes.

3. user (2026-06-21T07:56:57.449Z)

```
---BRANCH---
feat/serving-m1-deployment-profile
---STATUS---
M backend/config/models.py
M backend/main.py
M backend/storage/__init__.py
M backend/storage/ref.py
M tests/test_api_endpoints.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_input_source.py
```

4. assistant (2026-06-21T07:56:59.476Z)

Let me look at the diff against origin/master and read the key files.

5. user (2026-06-21T07:57:05.784Z)

```
9c64ac5 docs(serving): mark $7 tracker M1 ? + changelog (PR #279)
368155f feat(serving): M1 ? deployment.profile + compute_target config (pure, additive, default-OFF)
---DIFF STAT---
```

6. assistant (2026-06-21T07:57:08.482Z)

The diff against origin/master shows no backend changes committed yet ? the M2 changes are uncommitted (working tree). Let me diff the working tree.

7. user (2026-06-21T07:57:09.873Z)

```
backend/config/models.py      | 10 ++++++++
backend/main.py                | 33 ++++++++-----
backend/storage/__init__.py    | 45 ++++++++-----
backend/storage/ref.py         | 30 ++++++++
tests/test_api_endpoints.py    | 58 ++++++++
5 files changed, 165 insertions(+), 11 deletions(-)
```

8. user (2026-06-21T07:57:12.520Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 3612bd8..a31c9da 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -302,6 +302,16 @@ class StorageConfig(BaseModel):
     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
     # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
     output_root: str = ""
+    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
+    `backend`/`output_root`).
+    # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is read IN
```

```

PLACE
+ # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
+ # input name is resolved against ``input_root`` (e.g. ``input_root="gs://bucket/videos"`` +
"x.mp4"
+ # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
+ # (``"gs"`` + "bucket/x.mp4" => ``gs://bucket/x.mp4``). An explicit scheme on the input
+ # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs are
fetched to a
+ # scratch dir before processing (see ``backend.storage.acquire_local_input``).
+ input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
+ input_root: str = ""
+ # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
+ # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
+ # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the root
diff --git a/backend/main.py b/backend/main.py
index 942217e..33c7995 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -32,6 +32,7 @@ from backend.utils.paths import resolve_under_root, safe_output_filename,
valida
+ from backend.config import load_config as load_app_config, AppConfig, StitchingConfig # new
typed config
+ from backend.results import Failure # standardized error/result contract
+ from backend.reporting import emit_indexer_report
+from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)

app = FastAPI(title="Sports Highlight Indexer API")

@@ -151,13 +152,18 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
    notify = progress or (lambda stage: None)

    notify("hashing")
-    video_id = compute_video_id(req.video_path)
+ # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged (identity
?
+ # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
+ # req.video_path (the source URI) is kept for the DB record + report; only the file-reading
+ # consumers (hash / validators / segmenter) use the resolved local path.
+ local_video = storage.acquire_local_input(req.video_path, getattr(global_config, "storage",
None))
+ video_id = compute_video_id(local_video)
was_reingest = db_instance.get_video(video_id) is not None

+ # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the report)
notify("validating")
logger.info(f"Running validations for {req.video_path}...")
- val_results, val_context = registry.run_all_with_context(req.video_path, global_config)
+ val_results, val_context = registry.run_all_with_context(local_video, global_config)
failures = [r for r in val_results if not r.passed]

+ # typed AppConfig: attribute access for the default segmenter (with safe fallback)
@@ -205,7 +211,7 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
    segmenter = segmenter_cls(db_instance, global_config)
    try:
-        result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
+        result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
    except Exception:

```

```

        # A raising segmenter must not leave half-written rows: restore the pre-run
        # state (same destroy-after-confirm contract as the Failure branch), then let
@@ -265,8 +271,13 @@ def process_video(req: ProcessRequest):
    validate_input_path(req.video_path, allowed_root=allowed_root)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
-   if not os.path.exists(req.video_path):
-       raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}''")
+   # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive URI can't
be
+   # cheaply os.path.exists() 'd ? its existence is verified when the worker fetches it (a
fetch miss
+   # fails the job loudly). validate_input_path above still runs for every input: its null-
byte guard
+   # always applies, and a configured allowed_video_root (hosted mode) rejects a non-local
URI.
+   if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
+       if not os.path.exists(req.video_path):
+           raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}''")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
@@ -577,16 +588,20 @@ def main():
    # CLI processing mode (no web UI needed)
    if args.video_path and not args.server:
        print(f"[CLI] Processing video file: {args.video_path}")
-       if not os.path.exists(args.video_path):
+       # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
+       # The local-existence fast-fail only applies to a local input; a remote fetch fails
loudly.
+       storage_cfg = getattr(global_config, "storage", None)
+       if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
            print(f"[FAIL] Video file not found: {args.video_path}")
            return
+       local_video = storage.acquire_local_input(args.video_path, storage_cfg)

-       video_id = compute_video_id(args.video_path)
+       video_id = compute_video_id(local_video)
        was_reingest = db_instance.get_video(video_id) is not None

        # Validate (with-context variant surfaces ffprobe_meta for the report)
        print("[CLI] Validating...")
-       val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
+       val_results, val_context = registry.run_all_with_context(local_video, global_config)
        failures = [r for r in val_results if not r.passed]

    # Save video status
@@ -619,7 +634,7 @@ def main():
    segmenter_actual = seg_name
    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
    segmenter = segmenter_cls(db_instance, global_config)
-    result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
+    result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
    segmentation_ran = True

    if isinstance(result, Failure):
diff --git a/backend/storage/__init__.py b/backend/storage/__init__.py
index 78b4795..656274f 100644
--- a/backend/storage/__init__.py

```

```

+++ b/backend/storage/__init__.py
@@ -13,17 +13,58 @@ passthrough. See ``docs/DECOUPLED_COMPUTE``.
    from __future__ import annotations

    import os
+import tempfile
    from typing import Any, Dict, Iterator, Optional

    from backend.storage.base import StorageBackend
-    from backend.storage.ref import StorageRef, StorageStat, parse_uri
+    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri

    __all__ = [
-        "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
+        "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
+        "get_storage", "fetch", "put", "list_uris",
+        "acquire_local_input", "input_is_local",
    ]

+def _storage_dict(storage_config: Any) -> Dict[str, Any]:
+    """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
+    if storage_config is None:
+        return {}
+    if hasattr(storage_config, "model_dump"):
+        return storage_config.model_dump() # type: ignore[no-any-return]
+    return dict(storage_config)
+
+
+def input_is_local(raw: str, storage_config: Any = None) -> bool:
+    """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ? i.e. a
+    plain on-disk file the caller can stat/validate directly. False for a bucket / Drive URI
+    that must be fetched first. Lets callers keep their local-only checks (os.path.exists,
+    allowed_video_root) for the local case and skip them for a remote source."""
+    sc = _storage_dict(storage_config)
+    uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
+                           input_root=sc.get("input_root", ""))
+    return parse_uri(uri).backend == "local"
+
+
+def acquire_local_input(raw: str, storage_config: Any = None, *,
+                        scratch_parent: "str | None" = None) -> str:
+    """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for processing.
+
+    Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
    behaviour
+    byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a bare name
    under a
+    non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir and the
    local
+    path returned. The scratch copy is left in place (a pure intermediate, regenerable from the
+    source) ? same as the worker's ``cmd_infer`` pull, so cache-hygiene tooling reclaims it."""
+    sc = _storage_dict(storage_config)
+    uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
+                           input_root=sc.get("input_root", ""))
+    ref = parse_uri(uri)
+    if ref.backend == "local":
+        return ref.key # identity ? no temp, no copy, byte-identical to passing the path
    directly
+    scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
+    return fetch(uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
+
+
+def _backend_class(name: str):
+    """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""

```

```

    if name == "local":
diff --git a/backend/storage/ref.py b/backend/storage/ref.py
index ad5d784..60e2430 100644
--- a/backend/storage/ref.py
+++ b/backend/storage/ref.py
@@ -66,3 +66,33 @@ def parse_uri(uri: str) -> StorageRef:
     bucket, _, key = rest.partition("/")
     return StorageRef(scheme, bucket, key, s)
     raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
+
+
+def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "") -> str:
+    """Resolve a user-supplied input reference into a storage URI (COMPUTE_DECOUPLED_SERVING
M2).
+
+    Precedence (first match wins):
+    1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
+    2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always local).
+    3. A bare/relative name + a non-empty ``input_root`` ? joined under it
+    (``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
+    4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
``bucket/x`` ?
+    ``gcs://bucket/x``).
+    5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
+
+    So the default (``input_backend="local"`, ``input_root=""`) is an identity passthrough
for every
+    path used today, preserving byte-for-byte behaviour.
+    """
+    s = str(raw)
+    if _SCHEME_RE.match(s):          # 1. explicit scheme wins
+        return s
+    # 2. an anchored LOCAL path is always local, never joined. Detect this OS-INDEPENDENTLY ?
+    #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a POSIX-
style
+    #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows drive
prefix "X:".
+    if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
+        return s
+    if input_root:                  # 3. join a relative name under the configured root
+        return input_root.rstrip("/") + "/" + s.lstrip("/")
+    if input_backend and input_backend != "local":  # 4. prefix a bucket/key with the declared
backend
+        return f"{input_backend}://{s}"
+    return s                        # 5. bare/relative local path ? unchanged

```

9. assistant (2026-06-21T07:57:17.050Z)

Now let me read the full context of the key files. Let me read `ref.py` fully, the storage `\_\_init\_\_.py`, and the relevant parts of `main.py`.

10. user (2026-06-21T07:57:17.681Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5         <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6         local://<path>             -> local    (local:///abs or local://rel/p)
7         s3://<bucket>/<key>        -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8         gcs://<bucket>/<key>      -> gcs       (gs:// accepted, normalized to gcs://)
9         gdrive://<path>           -> gdrive    (path under your locally-MOUNTED Google

```

```

Drive "My Drive")
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14     from __future__ import annotations
15
16     import re
17     from dataclasses import dataclass
18     from datetime import datetime
19     from typing import Optional
20
21     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22     _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25     @dataclass(frozen=True)
26     class StorageStat:
27         """Lightweight object metadata (uniform across backends)."""
28         size: int
29         modified: Optional[datetime] = None
30         etag: Optional[str] = None
31         content_type: Optional[str] = None
32
33
34     @dataclass(frozen=True)
35     class StorageRef:
36         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38         backend: str
39         bucket: str
40         key: str
41         uri: str
42
43         @property
44         def name(self) -> str:
45             return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48     def parse_uri(uri: str) -> StorageRef:
49         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52         have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53         """
54         s = str(uri)
55         m = _SCHEME_RE.match(s)
56         if not m:
57             return StorageRef("local", "", s, f"local://{s}")
58         scheme = m.group(1).lower()
59         rest = s[m.end():]
60         if scheme == "gs":
61             scheme = "gcs"
62         if scheme in ("local", "gdrive"):
63             # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64             return StorageRef(scheme, "", rest, s)
65         if scheme in _KNOWN:
66             bucket, _, key = rest.partition("/")
67             return StorageRef(scheme, bucket, key, s)
68         raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")

```

```

69
70
71     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74         Precedence (first match wins):
75         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
(``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
78         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
``bucket/x`` ?
80         ``gcs://bucket/x``).
81         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83         So the default (``input_backend="local"`, ``input_root=""``) is an identity
passthrough for every
84         path used today, preserving byte-for-byte behaviour.
85         """
86         s = str(raw)
87         if _SCHEME_RE.match(s):          # 1. explicit scheme wins
88             return s
89         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90         #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91         #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
drive prefix "X:".
92         if s.startswith(("/", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93             return s
94         if input_root:                   # 3. join a relative name under the configured
root
95             return input_root.rstrip("/") + "/" + s.lstrip("/")
96         if input_backend and input_backend != "local":    # 4. prefix a bucket/key with the
declared backend
97             return f"{input_backend}://{s}"
98         return s                        # 5. bare/relative local path ? unchanged
99

```

11. user (2026-06-21T07:57:20.043Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13    from __future__ import annotations
14
15    import os
16    import tempfile

```



```

17     from typing import Any, Dict, Iterator, Optional
18
19     from backend.storage.base import StorageBackend
20     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22     __all__ = [
23         "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24         "get_storage", "fetch", "put", "list_uris",
25         "acquire_local_input", "input_is_local",
26     ]
27
28
29     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31         if storage_config is None:
32             return {}
33         if hasattr(storage_config, "model_dump"):
34             return storage_config.model_dump() # type: ignore[no-any-return]
35         return dict(storage_config)
36
37
38     def input_is_local(raw: str, storage_config: Any = None) -> bool:
39         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
40         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
41         that must be fetched first. Lets callers keep their local-only checks
(os.path.exists,
42         allowed_video_root) for the local case and skip them for a remote source."""
43         sc = _storage_dict(storage_config)
44         uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
45                               input_root=sc.get("input_root", ""))
46         return parse_uri(uri).backend == "local"
47
48
49     def acquire_local_input(raw: str, storage_config: Any = None, *,
50                            scratch_parent: "str | None" = None) -> str:
51         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
52
53         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
54         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
55         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
56         path returned. The scratch copy is left in place (a pure intermediate, regenerable
from the
57         source) ? same as the worker's ``cmd_infer`` pull, so cache-hygiene tooling reclaims
it."""
58         sc = _storage_dict(storage_config)
59         uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
60                               input_root=sc.get("input_root", ""))
61         ref = parse_uri(uri)
62         if ref.backend == "local":
63             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
64         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
65         return fetch(uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
66
67
68     def _backend_class(name: str):
69         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
70         if name == "local":
71             from backend.storage.local import LocalStorage

```

```

72         return LocalStorage
73     if name == "s3":
74         from backend.storage.s3 import S3Storage
75         return S3Storage
76     if name == "gcs":
77         from backend.storage.gcs import GCSStorage
78         return GCSStorage
79     if name == "gdrive":
80         from backend.storage.gdrive import GDriveStorage
81         return GDriveStorage
82     raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
83
84
85     def get_storage(backend: Optional[str] = None, *,
86                    config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
87         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
88         the implementation; its per-backend settings come from ``config[backend]`` merged
with
89         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
90         cfg = config or {}
91         name = backend or cfg.get("backend", "local")
92         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
93         settings.update(overrides)
94         return _backend_class(name)(**settings)
95
96
97     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
98             config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
99         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
100         (no copy); for s3/gcs it downloads to ``dst``."""
101         ref = parse_uri(uri)
102         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
103
104
105     def put(local_path: "str | os.PathLike[str]", uri: str, *,
106            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
107         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
108         ref = parse_uri(uri)
109         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
110         return uri
111
112
113     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
114         """Yield the URI of every object under ``prefix_uri``."""
115         ref = parse_uri(prefix_uri)
116         for r in get_storage(ref.backend, config=config).list(ref):
117             yield r.uri
118

```

12. assistant (2026-06-21T07:57:22.980Z)

Now let me look at the worker's reference pattern, the path-safety helpers, and how `fetch` handles `config` for non-local. Let me also check the local storage `fetch\_to\_local` for the identity-passthrough behavior, and how `config=sc` flows into `get\_storage`.

13. user (2026-06-21T07:57:24.108Z)

```

80     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
81         with open(path, "w") as f:
82             json.dump({
83                 "video_id": video_id,
84                 "status": status,
85                 "segment_count": len(segments),
86                 "segments": [{"start": float(s.get("start_time", 0.0)),
87                             "end": float(s.get("end_time", 0.0))} for s in segments],
88             }, f, indent=2)
89
90
91     def cmd_infer(args: argparse.Namespace) -> int:
92         config = load_config(args.config) if args.config else load_config()
93         _apply_overrides(config, args.set)
94         storage_cfg = config.storage.model_dump()
95
96         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
97         os.makedirs(scratch, exist_ok=True)
98         config.output.output_dir = scratch
99
100        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
101        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
102        print(f"[worker] pulled {args.video_uri} -> {local_video}")
103
104        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
105        video_id = compute_video_id(local_video)
106
107        # 3. VALIDATE (same registry as the main CLI).
108        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
109        failures = [r for r in val_results if not r.passed]
110        db = Database(os.path.join(scratch, config.output.db_name))
111        db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
112
113        segments: List[dict] = []
114        segmenter_actual = None
115        segmentation_ran = False
116        status = "failed"
117        try:
118            if failures:

```

14. assistant (2026-06-21T07:57:27.938Z)

Now let me check the local storage's `fetch\_to\_local` and how `config` is consumed by `get\_storage`. There's a subtle point: `acquire\_local\_input` passes `config=sc` where `sc` is `\_storage\_dict(storage\_config)`. But `get\_storage` expects `config[name]` for per-backend settings. Let me check.

15. user (2026-06-21T07:57:28.154Z)

```

1     """Local-filesystem storage backend ? the default, zero new dependencies.
2
3     Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4     file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5     the whole pipeline behaves exactly as before the storage layer existed.
6     """
7     from __future__ import annotations
8

```

```

9     import os
10    import shutil
11    from datetime import datetime
12    from typing import Iterator
13
14    from backend.storage.base import StorageBackend
15    from backend.storage.ref import StorageRef, StorageStat
16
17
18    class LocalStorage(StorageBackend):
19        def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20            # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21            self.base_dir = base_dir
22
23        def _path(self, ref: StorageRef) -> str:
24            p = ref.key
25            if self.base_dir and not os.path.isabs(p):
26                return os.path.join(self.base_dir, p)
27            return p
28
29        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30            # The object is already on the local FS ? return its path (identity). ``dst`` is
31            # honoured only if it names a *different* location (a copy is then made).
32            src = self._path(ref)
33            if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                return src
35            dst = str(dst)
36            if os.path.exists(dst) and not overwrite:
37                return dst
38            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39            shutil.copyfile(src, dst)
40            return dst
41
42        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43            dst = self._path(ref)
44            if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                shutil.copyfile(str(local_path), dst)
47            return ref
48
49        def exists(self, ref) -> bool:
50            return os.path.exists(self._path(ref))
51
52        def stat(self, ref) -> StorageStat:
53            st = os.stat(self._path(ref))
54            return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56        def list(self, prefix) -> Iterator[StorageRef]:
57            base = self._path(prefix)
58            if os.path.isdir(base):
59                for root, _dirs, files in os.walk(base):
60                    for fn in sorted(files):
61                        full = os.path.join(root, fn)
62                        yield StorageRef("local", "", full, f"local://{full}")
63            elif os.path.exists(base):
64                yield StorageRef("local", "", base, f"local://{base}")
65
66        def delete(self, ref) -> None:
67            path = self._path(ref)
68            if os.path.exists(path):
69                os.remove(path)
70

```

Now let me look at the path-safety helpers and the test files.

17. user (2026-06-21T07:57:30.953Z)

```
1      """Path-safety helpers for the API boundary (untrusted request fields)."""
2      import os
3      from typing import Optional
4
5
6      def safe_output_filename(name: str) -> str:
7          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
8
9          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
10         and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
11         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
12         """
13         if not name or not name.strip():
14             raise ValueError("output_filename must not be empty")
15         if "\x00" in name:
16             raise ValueError("output_filename must not contain a null byte")
17         if "/" in name or "\\" in name:
18             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
19         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
20             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
21         return name
22
23
24     def resolve_under_root(path: str, root: str) -> str:
25         """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
26         rp = os.path.realpath(path)
27         rr = os.path.realpath(root)
28         try:
29             contained = os.path.commonpath([rp, rr]) == rr
30         except ValueError: # different drives on Windows
31             contained = False
32         if not contained:
33             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34         return rp
35
36
37     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40         With no allowed_root (default, single-user local mode) only the null-byte check
applies,
41         so the local 'point it at any of my videos' workflow is preserved. Setting an
42         allowed_root (a future hosted/tenant deployment) enforces containment.
43         """
44         if "\x00" in path:
45             raise ValueError("video_path must not contain a null byte")
46         if allowed_root:
47             resolve_under_root(path, allowed_root)
48
```

18. assistant (2026-06-21T07:57:33.230Z)

Now the test files. Let me read the new test file and the API test changes.

19. user (2026-06-21T07:57:33.826Z)

```

1      """Input acquisition for COMPUTE_DECOUPLED_SERVING M2 ? resolve_input_uri /
input_is_local /
2      acquire_local_input.
3
4      The hard guarantee: the default config (input_backend='local', input_root='') is a byte-
for-byte
5      identity passthrough for every input path used today (no fetch, no copy). A bucket/Drive
source is
6      routed through storage.fetch to a local scratch file. All offline (the remote backend is
a
7      monkeypatched fetch ? no boto3/google-cloud-storage, no network).
8
9      OS note: os.path.isabs is platform-dependent, so the absolute-path cases use a POSIX-
style
10     '/data/...' (absolute on BOTH Windows and POSIX) and Windows backslash paths only with
input_root=''
11     (branch 5, OS-independent) ? the full suite runs cross-OS in CI.
12     """
13     import os
14
15     import pytest
16
17     from backend import storage
18     from backend.storage import acquire_local_input, input_is_local
19     from backend.storage.ref import resolve_input_uri
20
21
22     # ----- resolve_input_uri
23     @pytest.mark.parametrize("raw,ib,ir,expected", [
24         # 1. explicit scheme always wins (even over a non-local backend + root)
25         ("gs://b/x.mp4", "local", "", "gs://b/x.mp4"),
26         ("s3://b/x.mp4", "gcs", "gs://r", "s3://b/x.mp4"),
27         ("local://abs/x.mp4", "gcs", "gs://r", "local://abs/x.mp4"),
28         ("gdrive://Sharing/x.mp4", "local", "", "gdrive://Sharing/x.mp4"),
29         # 2. an anchored local path (leading slash, or drive prefix) is always local, never
joined ?
30         # OS-independently (not via os.path.isabs, which differs on Windows+py3.13)
31         ("/data/x.mp4", "gcs", "gs://r", "/data/x.mp4"),
32         ("\\\\server\\share\\x.mp4", "gcs", "gs://r", "\\\\server\\share\\x.mp4"),
33         ("C:\\v\\x.mp4", "gcs", "gs://r", "C:\\v\\x.mp4"),
34         ("C:/v/x.mp4", "gcs", "gs://r", "C:/v/x.mp4"),
35         # 3. a bare/relative name joins under a non-empty input_root
36         ("x.mp4", "local", "gs://bucket/videos", "gs://bucket/videos/x.mp4"),
37         ("sub/x.mp4", "local", "gs://bucket/videos/", "gs://bucket/videos/sub/x.mp4"), #
root trailing slash
38         # 4. input_backend prefixes a bare bucket/key when no input_root
39         ("bucket/x.mp4", "gcs", "", "gs://bucket/x.mp4"),
40         ("bucket/x.mp4", "s3", "", "s3://bucket/x.mp4"),
41         # 5. bare/relative local path ? today's behaviour, unchanged
42         ("x.mp4", "local", "", "x.mp4"),
43         ("rel/sub/x.mp4", "local", "", "rel/sub/x.mp4"),
44         ("C:\\v\\x.mp4", "local", "", "C:\\v\\x.mp4"), # Windows path, default config:
unchanged (OS-independent)
45     ])
46     def test_resolve_input_uri(raw, ib, ir, expected):
47         assert resolve_input_uri(raw, input_backend=ib, input_root=ir) == expected
48
49
50     def test_resolve_input_uri_default_is_identity():
51         for p in ("/data/clip.mp4", "rel/clip.mp4", "clip.mp4", "gs://b/k", "local://x"):
52             assert resolve_input_uri(p) == p
53
54
55     # ----- input_is_local
56     def test_input_is_local_true_for_local_paths():

```

```

57     assert input_is_local("/data/x.mp4") is True
58     assert input_is_local("rel/x.mp4") is True
59     assert input_is_local("local://x.mp4") is True
60
61
62     def test_input_is_local_false_for_buckets():
63         assert input_is_local("gs://b/x.mp4") is False
64         assert input_is_local("s3://b/x.mp4") is False
65         assert input_is_local("gdrive://Sharing/x.mp4") is False
66
67
68     def test_input_is_local_follows_input_root():
69         cfg = {"input_backend": "local", "input_root": "gs://bucket/videos"}
70         assert input_is_local("x.mp4", cfg) is False          # bare name resolves into the
bucket
71         assert input_is_local("/abs/x.mp4", cfg) is True      # absolute path stays local
72         assert input_is_local("gs://other/x.mp4", cfg) is False
73
74
75     def test_input_is_local_accepts_pydantic_storage_config():
76         from backend.config.models import StorageConfig
77         sc = StorageConfig(input_root="gs://bucket/videos")
78         assert input_is_local("x.mp4", sc) is False
79         assert input_is_local("/abs/x.mp4", sc) is True
80
81
82     # ----- acquire_local_input
83     def test_acquire_local_input_local_identity(tmp_path):
84         p = tmp_path / "v.mp4"
85         p.write_bytes(b"abc")
86         # Identity passthrough ? returns the SAME string, no copy, no temp dir.
87         assert acquire_local_input(str(p)) == str(p)
88
89
90     def test_acquire_local_input_relative_local_identity():
91         assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
92
93
94     def _capturing_fetch(captured):
95         def _f(uri, dst=None, *, config=None, overwrite=False):
96             captured["uri"] = uri
97             captured["dst"] = dst
98             captured["config"] = config
99             if dst:
100                 with open(dst, "wb") as fh:
101                     fh.write(b"remote-bytes")
102             return dst or uri
103         return _f
104
105
106     def test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
107         captured = {}
108         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
109         out = acquire_local_input("gs://bucket/videos/x.mp4", {"backend": "local", "gcs":
{}})
110         assert captured["uri"] == "gs://bucket/videos/x.mp4"          # exact URI routed to
fetch
111         assert os.path.basename(out) == "x.mp4"                      # named from the ref
112         assert os.path.isfile(out) and open(out, "rb").read() == b"remote-bytes"
113         # The whole storage settings dict is forwarded so per-backend (gcs/s3) config
reaches fetch.
114         assert captured["config"] == {"backend": "local", "gcs": {}}
115
116
117     def test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):

```

```

118         captured = {}
119         monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
120         acquire_local_input("x.mp4", {"input_root": "gs://bucket/videos"})
121         assert captured["uri"] == "gs://bucket/videos/x.mp4" # resolved BEFORE
fetch
122
123
124     def test_acquire_local_input_never_fetches_for_local(monkeypatch):
125         def _boom(*a, **k):
126             raise AssertionError("fetch must not be called for a local input")
127         monkeypatch.setattr(storage, "fetch", _boom)
128         assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
129         assert acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
== "clip.mp4"
130

```

20. assistant (2026-06-21T07:57:37.680Z)

There's an important detail in `test\_acquire\_local\_input\_remote\_fetches\_to\_scratch`. It monkeypatches `storage.fetch`, but `acquire\_local\_input` calls the module-level `fetch` directly. Let me verify whether the monkeypatch actually takes effect ? i.e. whether `acquire\_local\_input` references `fetch` via the module global (so `monkeypatch.setattr(storage, "fetch", ...)` works). Since `acquire\_local\_input` is defined in the same module and calls bare `fetch(...)` , it resolves `fetch` from the module globals at call time, so the monkeypatch works. Good.

Now let me check the API test changes.

21. user (2026-06-21T07:57:38.513Z)

```

diff --git a/tests/test_api_endpoints.py b/tests/test_api_endpoints.py
index 001c9fc..ebedddf 100644
--- a/tests/test_api_endpoints.py
+++ b/tests/test_api_endpoints.py
@@ -334,3 +334,61 @@ def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
     jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
     assert jobf["result"]["status"] == "failed"
     # empty-run similar (would preserve prior via reingest safety in both paths)
+
+
+## ----- M2: URI-aware input
+class _CaptureSeg:
+    """Records the video_path the segmenter actually received (M2: must be a LOCAL path)."""
+    last_path = None
+
+    def __init__(self, db, cfg):
+        self.db = db
+
+    def process_video(self, video_id, video_path, *a, **k):
+        _CaptureSeg.last_path = video_path
+        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
+        return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
{}}]
+
+
+def test_process_local_input_passes_identity_path(env):
+    """Default (local) input must reach the segmenter as the EXACT same path ? identity, no
copy
+    (the M2 zero-behaviour-change guarantee for today's local workflow)."""
+    client, db, tmp_path, mp = env
+    vid = _video(tmp_path)
+    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
+    _CaptureSeg.last_path = None
+    r = client.post("/api/process", json={"video_path": str(vid), "segmenter": "x"})

```



```

+     assert r.status_code == 202
+     assert _CaptureSeg.last_path == str(vid)
+
+
+def test_process_local_missing_file_still_404(env):
+     """A LOCAL input that doesn't exist still fast-fails 404 ? the existence gate is preserved
+     for local paths (only non-local URIs skip it)."""
+     client = env[0]
+     r = client.post("/api/process", json={"video_path": "/no/such/local/file.mp4"})
+     assert r.status_code == 404
+
+
+def test_process_nonlocal_uri_skips_exists_and_fetches(env):
+     """A gs:// input is NOT 404'd at submit (can't os.path.exists a bucket object); the worker
+     fetches it to a local scratch file and the segmenter sees that LOCAL path, not the URI."""
+     client, db, tmp_path, mp = env
+
+     def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+         with open(dst, "wb") as fh:
+             fh.write(b"fetched-video-bytes")
+         return dst
+
+     mp.setattr(m.storage, "fetch", fake_fetch)
+     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+     mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
+     _CaptureSeg.last_path = None
+     r = client.post("/api/process", json={"video_path": "gs://bucket/videos/x.mp4",
+ "segmenter": "x"})
+     assert r.status_code == 202                                # NOT 404 ? non-local skips the
exists gate
+     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
+     assert job["status"] == "done"
+     assert _CaptureSeg.last_path is not None
+     assert not _CaptureSeg.last_path.startswith("gs://")      # a fetched LOCAL path, not the
URI
+     assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"

```

22. assistant (2026-06-21T07:57:45.060Z)

Now let me look at the full `\_execute\_processing` and `process\_video` and the CLI block in main.py, plus the DB record write (to confirm provenance keeps the URI). Let me also check where `req.video\_path` is stored to the DB.

23. user (2026-06-21T07:57:45.573Z)

```

148     `progress` is an optional callable(stage: str) used to surface coarse job progress.
149     Raises on unexpected internal errors ? the caller records those as a job failure
150     (after this function has already restored the pre-run segments, see below).
151     """
152     notify = progress or (lambda stage: None)
153
154     notify("hashing")
155     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
156     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
157     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
158     # consumers (hash / validators / segmenter) use the resolved local path.
159     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
160     video_id = compute_video_id(local_video)
161     was_reingest = db_instance.get_video(video_id) is not None
162
163     # 1. Run pluggable validators (with-context variant surfaces ffmpeg_meta for the

```

```

report)
164     notify("validating")
165     logger.info(f"Running validations for {req.video_path}...")
166     val_results, val_context = registry.run_all_with_context(local_video, global_config)
167     failures = [r for r in val_results if not r.passed]
168
169     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
170     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
171     seg_name = req.segmenter or default_seg
172     segmenter_actual = None
173     segmentation_ran = False
174     response: Optional[Dict[str, Any]] = None
175     try:
176         if failures:
177             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
178             # status; it no longer destroys the video's prior good segments.
179             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
180             response = {
181                 "video_id": video_id,
182                 "status": "failed",
183                 "message": "Video failed validation checks.",
184                 "results": [r.model_dump() for r in val_results],
185             }
186             return response
187
188         # 2. Run segmenter & indexer
189         logger.info("[API] Video passed checks. Segmenting and indexing...")
190         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
191
192         segmenter_cls = segmenter_registry.get(seg_name)
193         if not segmenter_cls:
194             # Submit-time validation already 400s unknown names; this is the defensive
195             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
196             available = list(segmenter_registry.list_available().keys())
197             response = {
198                 "video_id": video_id,
199                 "status": "failed",
200                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
201                 "results": [r.model_dump() for r in val_results],
202                 "play_segments": [],
203             }
204             return response
205
206         logger.info(f"[API] Using segmenter: {seg_name}")
207         notify(f"segmenting ({seg_name})")
208         segmenter_actual = seg_name
209         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
210         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
211         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
212         segmenter = segmenter_cls(db_instance, global_config)
213         try:
214             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
215         except Exception:
216             # A raising segmenter must not leave half-written rows: restore the pre-run
217             # state (same destroy-after-confirm contract as the Failure branch), then
let
218             # the job worker record the failure.

```

```

219         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
220         raise
221     segmentation_ran = True
222
223     if isinstance(result, Failure):
224         logger.error(f"[API] Segmenter failed: {result.message}")
225         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
226         response = {
227             "video_id": video_id,
228             "status": "failed",
229             "message": f"Segmenter '{seg_name}' failed: {result.message}",
230             "results": [r.model_dump() for r in val_results],
231             "play_segments": [],
232         }
233         return response
234
235     segments = result.value if hasattr(result, "value") else result
236     notify("finalizing")
237     _finalize_reingest(video_id, segments, play_wm, cand_wm)
238
239     response = {
240         "video_id": video_id,
241         "status": "success",
242         "message": f"Successfully indexed {len(segments)} play segments using
243 '{seg_name}' segmenter.",
244         "results": [r.model_dump() for r in val_results],
245         "play_segments": segments,
246     }
247     return response
248
249     finally:
250         # Always emit the per-video observability report (next to the video, or to
251         # report.output_dir). Never raises. Surface the paths in the response.
252         paths = emit_indexer_report(
253             db=db_instance, video_id=video_id, video_path=req.video_path,
254             config=global_config,
255             val_results=val_results, val_context=val_context,
256             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
257             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
258         )
259         if paths and isinstance(response, dict):
260             response["reports"] = paths
261
262 @app.post("/api/process", status_code=202)
263 def process_video(req: ProcessRequest):
264     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
265
266     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
267     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
268     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
269     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
270     """
271     allowed_root = getattr(getattr(global_config, "security", None),
272 "allowed_video_root", None)
273     try:
274         validate_input_path(req.video_path, allowed_root=allowed_root)
275     except ValueError as e:
276         raise HTTPException(status_code=400, detail=str(e)) from e
277     # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive
278     # URI can't be
279     # cheaply os.path.exists()'d ? its existence is verified when the worker fetches it
280     # (a fetch miss
281     # fails the job loudly). validate_input_path above still runs for every input: its
282     # null-byte guard
283     # always applies, and a configured allowed_video_root (hosted mode) rejects a non-

```

local URI.

24. assistant (2026-06-21T07:57:53.783Z)

Now I have the full picture of `\_execute\_processing`. The DB record (lines 179, 190) and the report (line 251) use `req.video\_path` (the original URI) ? provenance preserved. The file-reading consumers (hash line 160, validators line 166, segmenter line 214) use `local\_video`. That's correct.

Let me check the validators ? do any validator read the path in a way that needs the local file? Specifically, I want to confirm validators read the file rather than just the path string. Let me check the ffprobe validator. But more importantly, let me examine a subtle concern: the `**report `emit_indexer_report`**` ? does it try to read the video file (e.g. ffprobe) using ``video_path`=URI`? If so, for a remote URI it would fail. Let me check.

25. user (2026-06-21T07:57:57.352Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 3612bd8..a31c9da 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -302,6 +302,16 @@ class StorageConfig(BaseModel):
===
```

26. user (2026-06-21T07:58:00.638Z)

```
<tool_use_error>Path does not exist: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting.py. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.</tool_use_error>
```

27. user (2026-06-21T07:58:03.484Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting

28. user (2026-06-21T07:58:06.430Z)

```
backend\reporting\__init__.py:27: def emit_indexer_report(
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-28- *,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-29- db,
backend\reporting\__init__.py-30- video_id: str,
backend\reporting\__init__.py-31- video_path: Optional[str],
backend\reporting\__init__.py-32- config: Optional[Dict[str, Any]],
backend\reporting\__init__.py-33- val_results: List[Any],
backend\reporting\__init__.py-34- val_context: Optional[Dict[str, Any]] = None,
backend\reporting\__init__.py-35- segmenter_requested: Optional[str] = None,
backend\reporting\__init__.py-36- segmenter_actual: Optional[str] = None,
backend\reporting\__init__.py-37- was_reingest: bool = False,
backend\reporting\__init__.py-38- segmentation_ran: bool = True,
backend\reporting\__init__.py-39-) -> Optional[Dict[str, str]]:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-40- """Harvest + write the report. Returns the written file paths, or None if
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-41- reporting is unavailable/disabled/failed. Never raises."""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-42- config
= config or {}
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-43- #
Accept either a plain dict or a typed config object (e.g. backend.config.AppConfig).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-44- #
Normalize to a plain nested dict so harvest's dict access works uniformly.
backend\reporting\__init__.py-45- if hasattr(config, "model_dump"):
backend\reporting\__init__.py-46-     try:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-47- config = config.model_dump(mode="json")
backend\reporting\__init__.py-48-     except Exception:
```

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-49-
config = {}
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-50-      #
Consume run signals unconditionally (even when reporting is off) so the
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-51-      #
process-local store never accumulates entries for finished runs.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-52-      signals
= run_signals.pop(video_id)
backend\reporting\__init__.py-53-      if not OBSREPORT_AVAILABLE or not report_enabled(config):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-54-
return None
backend\reporting\__init__.py-55-      try:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-56-
play_segments = db.query_play_segments(video_id, {}) if db is not None else []
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-57-
candidates = db.query_candidate_segments(video_id) if db is not None else []
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-58-      rec
= harvest.record_run(
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-59-
video_id=video_id,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-60-
video_path=video_path,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-61-
config=config,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-62-
val_results=val_results,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-63-
val_context=val_context,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-64-
play_segments=play_segments,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-65-
candidates=candidates,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-66-
segmenter_requested=segmenter_requested,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-67-
segmenter_actual=segmenter_actual,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-68-
was_reingest=was_reingest,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-69-
segmentation_ran=segmentation_ran,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-70-
run_signals=signals,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-71-      )
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-72-
output_dir = (config.get("report") or {}).get("output_dir")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-73-      res
= rec.write(output_dir=output_dir)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-74-
return res.paths or None
backend\reporting\__init__.py-75-      except Exception: # pragma: no cover - defensive;
reporting must never break a run
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\__init__.py-76-
return None

```

29. assistant (2026-06-21T07:58:10.151Z)

Let me check `harvest.record\_run` ? specifically whether it places the report file *next to the video* using `video\_path` (which for a remote URI would be `gs://...`, an invalid local dir). The `\_execute\_processing` comment at line 248 says "next to the video, or to report.output_dir". This is a potential gap for remote inputs ? but for the DEFAULT (local) config it's irrelevant. Let me confirm where the report is written for a remote URI.

30. user (2026-06-21T07:58:11.768Z)

```

backend\reporting\__init__.py:31: video_path: Optional[str],

```

```

backend\reporting\__init__.py:60:         video_path=video_path,
backend\reporting\__init__.py:72:         output_dir = (config.get("report") or
{}).get("output_dir")
backend\reporting\__init__.py:73:         res = rec.write(output_dir=output_dir)
backend\reporting\harvest.py:31:def video_meta_from_context(val_context: Optional[Dict[str,
Any]], video_path: Optional[str]) -> Dict[str, Any]:
backend\reporting\harvest.py:35:     if video_path and os.path.exists(video_path):
backend\reporting\harvest.py:37:         meta["size_mb"] = round(os.path.getsize(video_path)
/ (1024 * 1024), 1)
backend\reporting\harvest.py:180:     video_path: Optional[str],
backend\reporting\harvest.py:201:         video_path=video_path,
backend\reporting\harvest.py:205:     vm = video_meta_from_context(val_context, video_path)

```

31. user (2026-06-21T07:58:15.485Z)

```
---write---
```

32. user (2026-06-21T07:58:19.533Z)

```

__init__.py
__pycache__
harvest.py
run_signals.py
spec.yaml
---
```

33. user (2026-06-21T07:58:25.256Z)

```

4:and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
18:     import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
31:def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path: Optional[str])
-> Dict[str, Any]:
35:     if video_path and os.path.exists(video_path):
37:         meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
80:def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
93:     rec: "obsreport.RunRecorder",
137:def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
162:def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
180:     video_path: Optional[str],
194:) -> "obsreport.RunRecorder":
196:     rec = obsreport.RunRecorder(
201:         video_path=video_path,
205:     vm = video_meta_from_context(val_context, video_path)

```

34. assistant (2026-06-21T07:58:36.978Z)

The report's `video\_meta\_from\_context` (harvest.py:35) guards with `os.path.exists(video\_path)` ? so a `gs://` URI just yields no `size\_mb`, no crash. The report `.write()` is delegated to the `obsreport` external lib with `output\_dir` from config. For the default (local) config, the report still uses `req.video\_path` == the real local path, exactly as before. This is fine and provenance is preserved.

Now let me check the CLI block fully and the `--debug-clip`/`--trim` paths mentioned in part (d).

35. user (2026-06-21T07:58:37.686Z)

```

585         if swept:
586             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
587
588         # CLI processing mode (no web UI needed)
589         if args.video_path and not args.server:

```

```

590         print(f"[CLI] Processing video file: {args.video_path}")
591         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
592         # The local-existence fast-fail only applies to a local input; a remote fetch
593         # fails loudly.
594         storage_cfg = getattr(global_config, "storage", None)
595         if storage.input_is_local(args.video_path, storage_cfg) and not
596         os.path.exists(args.video_path):
597             print(f"[FAIL] Video file not found: {args.video_path}")
598             return
599         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
600
601         video_id = compute_video_id(local_video)
602         was_reingest = db_instance.get_video(video_id) is not None
603
604         # Validate (with-context variant surfaces ffprobe_meta for the report)
605         print("[CLI] Validating...")
606         val_results, val_context = registry.run_all_with_context(local_video,
607         global_config)
608         failures = [r for r in val_results if not r.passed]
609
610         # Save video status
611         status = "failed" if failures else "processed"
612         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
613         in val_results])
614
615         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
616         "default_segmenter", "motion")
617         segmenter_actual = None
618         segmentation_ran = False
619         try:
620             print("\n" + "="*40)
621             print("VALIDATION SUMMARY")
622             print("="*40)
623             for r in val_results:
624                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
625                 print(f"{status_symbol} {r.validator_name}: {r.message}")
626             print("="*40 + "\n")
627
628             if failures:
629                 print("[FAIL] Video failed checks. Indexing halted.")
630                 return
631
632         # Index
633         segmenter_cls = segmenter_registry.get(seg_name)
634         if not segmenter_cls:
635             print(f"[FAIL] Segmenter '{seg_name}' not found.")
636             return
637
638         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
639         segmenter_actual = seg_name
640         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
641         segmenter = segmenter_cls(db_instance, global_config)
642         result = segmenter.process_video(video_id, local_video,
643         max_frames=args.max_frames)
644         segmentation_ran = True
645
646         if isinstance(result, Failure):
647             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
648             print("[FAIL] Indexing halted for video.")
649             segments = []
650         else:
651             segments = result.value if hasattr(result, "value") else result
652             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
653             segments inside database: {db_path}")

```

```

647             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
648         _finalize_reingest(video_id, segments, play_wm, cand_wm)
649     finally:
650         # Always emit the per-video observability report. Never raises.
651         paths = emit_indexer_report(
652             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
653             val_results=val_results, val_context=val_context,
654             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,

```

36. assistant (2026-06-21T07:58:40.522Z)

The CLI block mirrors the API path correctly: file-reading uses `local\_video` (599, 604, 637), DB record + report use `args.video\_path` (609, 652). Now let me find the `--debug-clip`/`--trim` utility paths to assess part (d).

37. user (2026-06-21T07:58:42.281Z)

```

503:     parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video
clip to run detailed diagnostics on")
530:     if args.trim_start is not None and args.trim_end is not None:
531:         if not args.video_path:
536:             out_filename = args.trim_output or "trimmed_segment.mp4"
547:             print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}...")
556:             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
562:     if args.debug_clip is not None:
563:         if not args.video_path:
566:             run_cli_diagnose_clip(args.video_path, args.debug_clip)
589:     if args.video_path and not args.server:
590:         print(f"[CLI] Processing video file: {args.video_path}")
594:         if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
595:             print(f"[FAIL] Video file not found: {args.video_path}")
597:         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
609:         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])
652:         db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
664:     if args.server or not args.video_path:

```

38. user (2026-06-21T07:58:45.303Z)

```

528         return
529
530     if args.trim_start is not None and args.trim_end is not None:
531         if not args.video_path:
532             print("[ERROR] Please provide --video-path to extract a segment.")
533             return
534
535         # Determine output path
536         out_filename = args.trim_output or "trimmed_segment.mp4"
537         if os.path.isabs(out_filename):
538             out_path = out_filename
539             out_dir = os.path.dirname(out_path)
540             out_name = os.path.basename(out_path)
541         else:
542             out_dir = args.output_dir
543             out_path = os.path.join(out_dir, out_filename)
544             out_name = out_filename
545
546         os.makedirs(out_dir, exist_ok=True)

```



```

547         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
548
549         # global_config isn't initialized this early; load the typed config so the trim
550         # clip carries the same configured stitching paddings + watermark as a real
551         # highlight (watermark default-on).
552         stitching = load_app_config().stitching
553         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
554                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
555         try:
556             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
557             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
558         except Exception as e:
559             print(f"[ERROR] Failed to compile trimmed segment: {e}")
560         return
561
562     if args.debug_clip is not None:
563         if not args.video_path:
564             print("[ERROR] Please provide --video-path to debug a specific clip.")
565         return
566     run_cli_diagnose_clip(args.video_path, args.debug_clip)
567     return

```

39. assistant (2026-06-21T07:58:51.335Z)

The `--trim/--debug-clip` utility paths (lines 556, 566) read `args.video_path` directly. For the default local config, these are byte-identical to before (no behaviour change), so the M2 zero-behaviour-change guarantee holds. For a remote URI they would not fetch ? but these are diagnostic/dev utilities, and M2's spec scope is "main CLI/API input." This is an acceptable scope boundary, not a correctness gap for M2. I'll note it as a nit.

Now let me verify a key correctness concern in `acquire_local_input`: it passes `config=sc` to `fetch`. `fetch` ? `get_storage(ref.backend, config=config)` ? `settings = dict(cfg.get(name, {}))` or `{}`. So for `gcs`, it pulls `sc.get("gcs", {})`. The `StorageConfig.model_dump()` must contain a `gcs` key for per-backend settings to flow. Let me check `StorageConfig` has nested backend config keys.

40. user (2026-06-21T07:58:52.052Z)

```

280         # mode). Default None preserves the single-user local 'any local file' workflow.
281         allowed_video_root: Optional[str] = None
282
283         # --- Chunked upload (B2, PR-2) ---
284         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286         # contain upload_dir or /api/process will reject the uploaded paths.
287         upload_dir: str = "output/uploads"
288         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
290         max_upload_mb: int = 4096
291         max_part_mb: int = 95
292         allowed_upload_extensions: List[str] = ["mp4", "mov"]
293
294
295         class StorageConfig(BaseModel):
296             """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297             behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
298             + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299             Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
300             **secrets are never stored here**, only endpoints / regions / file paths / source-

```

```

selectors
301      (boto3/google auth chains supply credentials). See ``backend/storage/``.
302      backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303      # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304      output_root: str = ""
305      # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
306      # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
307      # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
308      # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
309      # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
310      # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
311      # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
312      # scratch dir before processing (see ``backend.storage.acquire_local_input``).
313      input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
314      input_root: str = ""
315      # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
316      # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
317      # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
318      # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
319      # the legacy flat layout is used unchanged.
320      single_folder_per_video: bool = False
321      # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
322      workspace_root: str = ""
323      # Cloud namespace prefix under the root so per-video folders sit tidily beside
324      # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
325      workspace_prefix: str = "workspaces"
326      local: dict[str, Any] = Field(default_factory=dict)
327      s3: dict[str, Any] = Field(default_factory=dict)
328      gcs: dict[str, Any] = Field(default_factory=dict)
329      gdrive: dict[str, Any] = Field(default_factory=dict)
330
331
332      class DeploymentConfig(BaseModel):
333          """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
334          knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
335
336          **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
337          behaves exactly as before this section existed. A profile is opt-in (``config.json``
338          ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
339          *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads

```

41. assistant (2026-06-21T07:58:55.568Z)

`StorageConfig` carries the nested `local/s3/gcs/gdrive` dicts (326-329), and `model\_dump()` includes them. So `fetch(config=sc)` correctly receives `sc["gcs"]` etc. Good ? per-backend settings flow.

Now let me run the new tests and the API tests to confirm green, plus verify the byte-identity claims empirically.

42. user (2026-06-21T07:58:58.527Z)

..... [100%]
25 passed in 0.33s

43. user (2026-06-21T07:59:14.611Z)

..... [100%]
17 passed in 10.82s

44. assistant (2026-06-21T07:59:16.842Z)

Now let me empirically verify the byte-identity claims for the four path forms mentioned in the lens (part a), plus the `input\_is\_local` identity check and the `acquire\_local\_input` identity object behavior.

45. user (2026-06-21T07:59:22.119Z)

```
Exit code 1
  File "<string>", line 5
    cases = ['C:\v\x.mp4', 'x.mp4', 'C:/v/x.mp4', '/data/x.mp4', 'rel/sub/x.mp4', 'local://x']
            ^^^^^^^^^^^^^^^^^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 4-5: truncated
\xXX escape
```

46. assistant (2026-06-21T07:59:24.044Z)

Let me write this to a file to avoid shell-quoting hell with backslashes.

47. assistant (2026-06-21T07:59:28.827Z)

```
from backend.storage.ref import resolve_input_uri as r
from backend.storage import acquire_local_input as a, input_is_local as il

cases = ["C:\\v\\x.mp4", "x.mp4", "C:/v/x.mp4", "/data/x.mp4", "rel/sub/x.mp4"]
print("--- resolve_input_uri default (identity?) ---")
for c in cases:
    out = r(c)
    print(repr(c), "->", repr(out), "IDENTITY" if out == c else "CHANGED!!!")

print("--- acquire_local_input returns SAME value (and same object?) ---")
for c in cases:
    out = a(c)
    print(repr(c), "->", repr(out), "SAME" if out == c else "DIFF",
          "isSameObj" if out is c else "newStr")

print("--- local:// scheme via acquire (what local path comes out?) ---")
print("local://x.mp4 ->", repr(a("local://x.mp4")))
print("local:///abs/x.mp4 ->", repr(a("local:///abs/x.mp4")))

print("--- input_is_local default ---")
for c in cases + ["gs://b/x", "local://x"]:
    print(repr(c), il(c))
```

48. user (2026-06-21T07:59:28.842Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\scratch_m2_check.py (file state is current in your context ? no need to Read it back)

49. user (2026-06-21T07:59:31.692Z)

```
Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\_m2_check.py", line 1, in
```

```
<module>
    from backend.storage.ref import resolve_input_uri as r
ModuleNotFoundError: No module named 'backend'
```

50. user (2026-06-21T07:59:34.501Z)

```
--- resolve_input_uri default (identity?) ---
'C:\v\X.mp4' -> 'C:\v\X.mp4' IDENTITY
'x.mp4' -> 'x.mp4' IDENTITY
'C:/v/x.mp4' -> 'C:/v/x.mp4' IDENTITY
'/data/x.mp4' -> '/data/x.mp4' IDENTITY
'rel/sub/x.mp4' -> 'rel/sub/x.mp4' IDENTITY
--- acquire_local_input returns SAME value (and same object?) ---
'C:\v\X.mp4' -> 'C:\v\X.mp4' SAME isSameObj
'x.mp4' -> 'x.mp4' SAME isSameObj
'C:/v/x.mp4' -> 'C:/v/x.mp4' SAME isSameObj
'/data/x.mp4' -> '/data/x.mp4' SAME isSameObj
'rel/sub/x.mp4' -> 'rel/sub/x.mp4' SAME isSameObj
--- local:// scheme via acquire (what local path comes out?) ---
local://x.mp4 -> 'x.mp4'
local:///abs/x.mp4 -> '/abs/x.mp4'
--- input_is_local default ---
'C:\v\X.mp4' True
'x.mp4' True
'C:/v/x.mp4' True
'/data/x.mp4' True
'rel/sub/x.mp4' True
'gs://b/x' False
'local://x' True
```

51. assistant (2026-06-21T07:59:41.306Z)

Confirmed: under the default config, `acquire\_local\_input` returns the **exact same string object** (`isSameObj`) for a bare path or for a plain bare path `resolve\_input\_uri` returns `s` unchanged, but note `s = str(raw)`. When `raw` is already a `str`, `str(raw)` returns the same object, so identity holds. Byte-identity verified for all four lens path forms.

One subtlety worth noting: for an explicit `local://x.mp4` URI, `acquire\_local\_input` returns `ref.key` = `"x.mp4"` (scheme stripped). That's a *new capability* (not used today by default), and it's correct? `local://` is documented as a local path. Not a regression since today's callers pass bare paths, not `local://`.

Now let me verify part (b)? did the wiring miss any file-reading use of `req.video\_path`/`args.video\_path` that still passes the URI? Let me grep all uses in `\_execute\_processing` and the CLI block.

52. user (2026-06-21T07:59:42.331Z)

```
157:     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
159:     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
160:     video_id = compute_video_id(local_video)
165:     logger.info(f"Running validations for {req.video_path}...")
166:     val_results, val_context = registry.run_all_with_context(local_video, global_config)
179:         db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump() for r
in val_results])
190:         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for r
in val_results])
214:         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
251:         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
271:         validate_input_path(req.video_path, allowed_root=allowed_root)
```

```

278:     if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
279:         if not os.path.exists(req.video_path):
280:             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
289:     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
531:         if not args.video_path:
547:         print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}...")
556:         res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
563:         if not args.video_path:
566:         run_cli_diagnose_clip(args.video_path, args.debug_clip)
589:     if args.video_path and not args.server:
590:         print(f"[CLI] Processing video file: {args.video_path}")
594:         if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
595:             print(f"[FAIL] Video file not found: {args.video_path}")
597:         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
599:         video_id = compute_video_id(local_video)
604:         val_results, val_context = registry.run_all_with_context(local_video, global_config)
609:         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])
637:         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
652:         db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
664:     if args.server or not args.video_path:

```

53. assistant (2026-06-21T07:59:49.928Z)

This is the critical part. Let me classify every use:

****File-reading consumers (must use `local\_video`):**** 160 (hash), 166 (validators), 214 (segmenter) ? all use `local\_video`. CLI: 599, 604, 637 ? all use `local\_video`. ?

****Provenance/record (must keep URI):**** 179, 190 (DB add_video), 251 (report), 289 (create_job), 609 (CLI DB), 652 (CLI report). ? All keep `req.video\_path`/`args.video\_path`.

****Now the key question ? line 289 `create\_job`.**** The API `/api/process` is async: it creates a job with `req.video\_path` (the URI), and the *worker* later runs `\_execute\_processing`. So the job stores the URI, and the worker fetches it. Good ? that's the intended flow. The test `test\_process\_nonlocal\_uri\_skips\_exists\_and\_fetches` confirms the gs:// job runs to "done" and the segmenter sees a local path.

But wait ? there's a subtle gap. Line 289 `create\_job` stores `req.video\_path` (the URI). Let me check whether the job worker that drains the queue calls `\_execute\_processing(req)` reconstructing `req.video\_path` from the stored value, OR whether there's a *separate* `os.path.exists` / validation gate in the job-draining path that would re-reject a non-local URI. Let me trace the job worker.

54. user (2026-06-21T07:59:51.319Z)

```

281         if req.segmenter and not segmenter_registry.get(req.segmenter):
282             available = list(segmenter_registry.list_available().keys())
283             raise HTTPException(
284                 status_code=400,
285                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
286             )
287
288         job_id = uuid.uuid4().hex
289         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
290             req.player_pool, req.max_frames):
291             raise HTTPException(status_code=500, detail="Failed to persist job record.")
292         _get_executor().submit(_drain_due_jobs)

```

```

293     return JsonResponse(
294         status_code=202,
295         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
296     )
297
298
299 @app.get("/api/jobs/{job_id}")
300 def get_job(job_id: str):
301     """Job state: {status, progress, result, reports}. `status` = execution state
302     (queued|running|done|failed); the pipeline verdict is result["status"]."""
303     job = db_instance.get_job(job_id)
304     if not job:
305         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
306     result = job.get("result")
307     return {
308         "job_id": job["id"],
309         "status": job["status"],
310         "progress": job.get("progress"),
311         "video_id": job.get("video_id"),
312         "error": job.get("error"),
313         "result": result,
314         "reports": (result or {}).get("reports"),
315         "created_at": job.get("created_at"),
316         "started_at": job.get("started_at"),
317         "finished_at": job.get("finished_at"),
318     }
319
320 # --- Chunked upload (B2, PR-2) -----
321 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
322 # in
323 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
324 # via
325 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
326 # sequential-part logic, and abort cleanup ? is unchanged.
327 from backend.api import uploads as uploads_api
328
329 app.include_router(uploads_api.router)
330
331 # --- Highlight download (B3, PR-2) -----
332
333 @app.get("/api/highlights/{video_id}/{filename}")
334 def download_highlight(video_id: str, filename: str):
335     """Stream a compiled highlight reel ? `filename` is the bare name given to
336     /api/compile
337     (which only writes into output_dir; the browser previously got back an unreachable
338     server-local path)."""
339     try:
340         name = safe_output_filename(filename)
341     except ValueError as e:
342         raise HTTPException(status_code=400, detail=str(e)) from e

```

55. assistant (2026-06-21T07:59:53.820Z)

Let me trace `\_drain\_due\_jobs` to confirm it reconstructs `ProcessRequest` from the stored URI and calls `\_execute\_processing` without any extra existence gate.

56. user (2026-06-21T07:59:55.500Z)

```

57:     if os.path.exists(index_path):
58:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
59:     _drain_due_jobs,
60:
61:class ProcessRequest(BaseModel):
62:
63:     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
64:
65:     # cheaply os.path.exists() ? its existence is verified when the worker fetches it (a

```

```

fetch miss
279:         if not os.path.exists(req.video_path):
292:     _get_executor().submit(_drain_due_jobs)
594:         if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
669:         _get_executor().submit(_drain_due_jobs)

```

57. assistant (2026-06-21T07:59:58.062Z)

`\_drain\_due\_jobs` is imported from `backend.jobs` (line 76). Let me find it and confirm it reconstructs the request and calls `\_execute\_processing` with no extra exists-gate.

58. user (2026-06-21T08:00:00.214Z)

```

Exit code 2
===

```

59. user (2026-06-21T08:00:03.808Z)

```

backend/api/job_worker.py:9:every cross-function seam ? `db_instance`, `global_config`,
`_execute_processing`,
backend/api/job_worker.py:46:     is the segmenter's `_execute_processing`, run UNDER the
destroy-after-confirm contract.
backend/api/job_worker.py:60:     """Reconstruct the submit-time ProcessRequest from a claimed
job row, so a job
backend/api/job_worker.py:62:     the original submit. The row stores exactly the ProcessRequest
fields."""
backend/api/job_worker.py:64:     return _main.ProcessRequest(
backend/api/job_worker.py:86:         result = _main._execute_processing(
backend/api/job_worker.py:100:def _drain_due_jobs() -> int:
backend/main.py:91:class ProcessRequest(BaseModel):
backend/main.py:140:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
backend/main.py:261:def process_video(req: ProcessRequest):

```

60. user (2026-06-21T08:00:06.565Z)

```

55         self.delay_sec = max(0.0, float(delay_sec))
56         self.progress = progress
57
58
59     def _job_to_request(job: Dict[str, Any]):
60         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62         the original submit. The row stores exactly the ProcessRequest fields."""
63         import backend.main as _main
64         return _main.ProcessRequest(
65             video_path=job["video_path"],
66             player_pool=job.get("player_pool"),
67             max_frames=job.get("max_frames"),
68             segmenter=job.get("segmenter"),
69         )
70
71
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
74         its
75         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
76         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
77         means
78         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
79         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
80         the

```

```

79     job self-scheduled and will be re-claimed when `next_poll_at` is due.
80     """
81     import backend.main as _main
82     db_instance = _main.db_instance
83     job_id = job["id"]
84     req = _main.job_to_request(job)
85     try:
86         result = _main._execute_processing(
87             req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88         if result.get("video_id"):
89             db_instance.set_job_video_id(job_id, result["video_id"])
90             db_instance.mark_job_done(job_id, result)
91     except _main.JobWaiting as w:
92         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
93         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
94         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
95     except Exception as e:
96         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
97         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
98
99
100     def _drain_due_jobs() -> int:
101         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
102         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
103
104         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
105         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
106         a single-worker box keeps making progress with no central timer (the DB is the
clock).
107         Returns the number of jobs that reached a terminal/parked state this tick.
108         """
109         import backend.main as _main

```

61. assistant (2026-06-21T08:00:10.550Z)

The job-draining path reconstructs `ProcessRequest(video\_path=job["video\_path"])` (the stored URI) and calls `\_execute\_processing` ? which does the acquire-then-use. No extra `os.path.exists` gate in the worker path. So a gs:// URI flows through correctly. The wiring is complete and correct.

Now part (c): confirm `compute\_video\_id` hashes the FETCHED bytes for remote and the same local file for local. Let me check `compute\_video\_id`.

62. user (2026-06-21T08:00:11.439Z)

```

1     """Canonical file-identity hashing for the indexer.
2
3     `video_id` is the MD5 of the *whole* video file. Historically each endpoint
4     (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
5     defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6     The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7     any future change to one copy would silently diverge the DB key.
8
9     This module is the single source of truth. Import `compute_video_id` everywhere a
10    video needs to be keyed.
11    """
12
13    from __future__ import annotations
14
15    import hashlib
16

```



```

17     # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18     # was centralized. The chunk size does not affect the resulting digest ? it only
19     # bounds memory use while streaming large files.
20     _CHUNK_SIZE = 65536
21
22
23     def compute_video_id(filepath: str) -> str:
24         """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26         Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27         into memory at once.
28         """
29         hasher = hashlib.md5()
30         with open(filepath, "rb") as f:
31             for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                 hasher.update(chunk)
33         return hasher.hexdigest()
34

```

63. assistant (2026-06-21T08:00:24.766Z)

`compute\_video\_id` opens and hashes the actual file at `filepath`. In `\_execute\_processing` line 160 and CLI line 599, `filepath=local\_video` ? the fetched local copy for remote, the in-place file for local. So:

- Local: hashes the same local file as before ? identity unchanged. ?
- Remote: hashes the **fetched bytes** ? correct content-identity. This matches the worker's pattern (worker/`\_\_main\_\_`.py:104-105 comment: "pulled bytes must be byte-identical"). ?

This is correct. Note a desirable property: since `video\_id` is content-MD5, a remote `gs://.../x.mp4` and a local copy of the same bytes produce the **same** `video\_id` ? consistent identity. Good.

Now let me check one thing about the `acquire\_local\_input` scratch-fetch for remote: `ref.name` or "in.mp4". For `gs://bucket/videos/x.mp4`, `parse\_uri` gives `bucket="bucket"`, `key="videos/x.mp4"`, so `ref.name` = `"x.mp4"`. Good. But what about a key ending in `/` or a bucket-root object? Edge case, not a default-config concern.

Let me check whether there's a concern with the worker path (`backend/worker/`__main__`.py`) ? does M2 leave it consistent, or does the worker now use a **different** resolution than the API? The worker uses `storage.fetch(args.video\_uri, ...)` directly (line 101), NOT `acquire\_local\_input`/`resolve\_input\_uri`. That means the worker does NOT honor `input\_root`/`input\_backend`. Is that a gap? The worker takes a full `video\_uri` (already a URI), so `input\_root` join isn't needed there ? it's a different contract (explicit URI in). Acceptable. Let me note this as informational consistency.

Let me now verify the `input\_is\_local` + `validate\_input\_path` interaction in hosted mode (the comment at main.py:276 claims `allowed\_video\_root` rejects a non-local URI). Let me verify that claim, since it's a security-relevant assertion.

64. user (2026-06-21T08:00:27.848Z)

```

gs:// with allowed_root: REJECTED -> path 'gs://bucket/videos/x.mp4' escapes the allowed root
'/srv/videos'
cwd C:\Users\avidu\Projects\badminton-highlight-indexer

```

65. assistant (2026-06-21T08:00:34.438Z)

The hosted-mode claim holds: with `allowed\_video\_root` set, a `gs://` URI is rejected by `validate\_input\_path` (it resolves to a path that escapes the root). So a non-local URI cannot be used in hosted mode ? which is the intended behaviour (remote inputs are an explicit single-tenant/local-deploy feature, not exposed to untrusted hosted callers).

Now let me double-check one potential concern flagged by the lens part (b): "replace a use that must stay the URI." Lines 165 (`logger.info(f"Running validations for {req.video\_path}")`) keeps

the URI for the log message ? that's fine/desirable (logs the source). All provenance uses keep the URI. No misclassification found.

Let me run the broader storage test suite and worker test to ensure no regression, then clean up the scratch file.

66. user (2026-06-21T08:00:47.937Z)

```
..... [ 57%]
..... [100%]
126 passed, 970 deselected in 10.06s
```

67. assistant (2026-06-21T08:00:52.776Z)

All 126 storage/worker/input/ref tests pass. Let me do a final check on one nuance: the `acquire\_local\_input` for a remote source leaves the scratch copy in place (the docstring claims cache-hygiene tooling reclaims it). Let me verify the scratch dir prefix `rally-input-` is actually covered by the cleanup tooling, since the docstring makes that claim ? if not, it's a leak claim that's false (attribution discipline).

68. user (2026-06-21T08:00:55.409Z)

```
=== search for the cleanup tool ===
tools/cleanup_caches.py
```

69. user (2026-06-21T08:00:58.922Z)

```
50:         if low.startswith("tmp"):
```

70. user (2026-06-21T08:01:01.567Z)

```
30     from typing import List, Optional, Tuple
31
32     # ---- classification (pure; unit-tested) ----- #
33
34     # (category, regenerable, default_purge). default_purge=True ? deleted unless a flag
35     # protects it. wasbcache is regenerable but default_purge=False (it enables resume and
36     # is tiny) ? opt in with --include-wasbcache.
37     _KEEP = ("keep", False, False)
38
39
40     def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
41         """Classify a cache entry by name. Returns (category, regenerable,
default_purge)."""
42         low = name.lower()
43         if is_dir:
44             if low.endswith("_frames"):
45                 return ("frame cache", True, True)
46             if low.endswith("_wasbcache"):
47                 return ("detector resume cache", True, False) # opt-in via --include-
wasbcache
48             if low.startswith("chunks_") and low.endswith("_handoff"):
49                 return ("ai handoff clips", True, True)
50             if low.startswith("tmp"):
51                 return ("temp dir", True, True)
52             return ("other dir", False, False)
53         # files
54         if low.endswith((".csv",)):
55             return ("trajectory csv", False, False) # durable analysis output
56         if low.endswith((".db", ".sqlite", ".sqlite3")):
57             return ("database", False, False) # cached rallies ? never purge
58         if low.endswith((".log", ".sh", ".txt", ".json", ".md")):
59             return ("log/script", False, False)
60         if low.endswith((".mp4", ".avi", ".mov", ".mkv")):
```

```

61         # In the WSL stage dir, every video is a regenerable staged copy.
62         if location == "wsl":
63             return ("staged video", True, True)
64         # In output/, a 1080p/proxy is regenerable; anything else is a produced reel.
65         if "_1080p" in low or "_proxy" in low or low.endswith("_proxy.mp4"):
66             return ("proxy video", True, True)
67         return ("highlight reel", False, False)          # durable output ? keep
68     return ("other", False, False)
69

```

71. assistant (2026-06-21T08:01:12.679Z)

The cleanup tool classifies dirs starting with ``tmp`` as temp dirs (line 50), but ``acquire_local_input`` uses ``mkdtemp(prefix="rally-input-")`` ? and the `*worker*` uses ``prefix="rally-worker-")``. Neither starts with ``tmp``. More importantly, ``tempfile.mkdtemp(dir=None)`` places the dir under the `**system temp**` (``%TEMP%`` / ``/tmp``), not under any scan root that ``cleanup_caches.py`` walks. So the docstring claim "the scratch copy is left in place ... so cache-hygiene tooling reclaims it" is `**not**` substantiated by ``cleanup_caches.py`` ? the tool neither walks the system temp dir nor matches the ``rally-input-`` prefix.

However, this is the `*same pattern the worker already ships*` (worker leaves ``rally-worker-`` scratch under system temp too), and it only affects the `*non-default*` (remote-input) path ? under the default local config, no scratch is created at all (identity passthrough). The OS reclaims ``/tmp`` on reboot. So this is a minor doc-accuracy / latent-leak nit on the non-default path, not a behaviour-change blocker for M2. I'll flag it as minor with the attribution-discipline angle (the docstring asserts a reclaim path that isn't wired).

Let me do one last verification of the lens's exact four path forms through ``input_is_local`` under a `*non-default*` config to make sure the absolute-path escape hatch (branch 2) protects local Windows paths even when a remote ``input_root`` is set (so a user with ``input_root=gs://...`` configured can still process a local file). Already covered by ``test_input_is_local_follows_input_root`` (``/abs/x.mp4`` ? True). Confirmed.

Let me verify the full test suite collects cleanly (no import errors introduced) by running a quick collection-only pass.

72. user (2026-06-21T08:01:23.502Z)

```

tests/test_yolo_hybrid.py::test_extract_windows_returns_enriched_dicts
tests/test_yolo_hybrid.py::test_yolo_hybrid_persists_and_links_candidates
tests/test_yolo_hybrid.py::test_yolo_hybrid_store_candidates_disabled

```

1096 tests collected in 7.76s

73. assistant (2026-06-21T08:01:25.988Z)

1096 tests collect cleanly. Let me run the M0 config/models tests too, to make sure the new ``input_backend``/``input_root`` fields don't break any config validation.

74. user (2026-06-21T08:01:43.961Z)

```

..... [ 64%]
..... [100%]
111 passed, 985 deselected in 14.71s

```

75. assistant (2026-06-21T08:01:52.349Z)

All green. I have completed my review across all four lens sub-questions. Let me compile the findings.

Summary of verification:

- `**a)` byte-identical local: Empirically confirmed ``resolve_input_uri`` and ``acquire_local_input`` return the `*same string object*` for ``C:\v\x.mp4``, ``x.mp4``, ``C:/v/x.mp4``,

`/data/x.mp4`, `rel/sub/x.mp4` under default config. No normalization, no copy.

- **(b) wiring completeness:** Every file-reading consumer (hash/validators/segmenter) in both `\_execute\_processing` and the CLI block uses `local\_video`; every provenance use (DB `add\_video`, `create\_job`, report) keeps the URI. The async job-worker path reconstructs the request from the stored URI and runs identically. No missed file-reading use, no over-replaced provenance use.
- **(c) compute_video_id:** Hashes `local\_video` ? the fetched bytes for remote (correct content-identity, matching worker semantics), the same in-place file for local (unchanged).
- **(d) --debug-clip/--trim:** Use `args.video\_path` directly; acceptable scope boundary (diagnostic utilities, byte-identical under default config).

Two minor issues found (neither a blocker nor a default-config behaviour change).

76. user (2026-06-21T08:02:14.225Z)

Structured output provided successfully

77. assistant (2026-06-21T08:02:26.770Z)

Review complete. M2 of COMPUTE_DECOUPLED_SERVING is a clean, safe, default-OFF change.

Verdict: SAFE / byte-identical-local guarantee holds. Two findings, neither a blocker.

Key verification results:

- **(a) byte-identical local** ? Confirmed empirically that under the default config (`input\_backend='local', input\_root=''), `resolve_input_uri` and `acquire_local_input` return the *same string object* (not just equal value) for all four lens path forms (`C:\v\x.mp4`, `x.mp4`, `C:/v/x.mp4`, `/data/x.mp4`) plus relative paths. No normalization, no copy, no temp dir. The OS-independent anchored-path detection (`backend/storage/ref.py:92`) correctly handles a POSIX `/data/x` on Windows+py3.13 where `os.path.isabs` would fail.
- **(b) wiring** ? No leaks, no over-replacement. File-reading consumers (hash/validators/segmenter) all use `local\_video`; provenance (DB record, job row, report) all keep the original URI. The async job-worker path (`backend/api/job\_worker.py:64-69`) reconstructs the request from the stored URI and runs `\_execute\_processing` with no extra existence gate, so a `gs://` job fetches-then-processes correctly.
- **(c) compute_video_id** ? Hashes the fetched bytes for remote (correct content-identity, matching the worker at `backend/worker/\_\_main\_\_.py:104-105`) and the unchanged in-place file for local.
- **(d) --trim / --debug-clip** ? Use `args.video\_path` directly; acceptable scope boundary (dev utilities, byte-identical under default config).

Tests: `tests/test\_input\_source.py` (25) + the 3 new API tests all pass; 126 storage/worker tests green; full suite of 1096 collects clean.

The one **minor** finding worth addressing before merge: the `acquire\_local\_input` docstring (`backend/storage/\_\_init\_\_.py:56-57`) claims "cache-hygiene tooling reclaims" the remote scratch dir, but `tools/cleanup\_caches.py` neither scans the system temp dir nor matches the `rally-input-` prefix it actually uses ? a latent leak on the non-default remote path and a docstring-accuracy issue (pre-existing pattern, shared with the worker's `rally-worker-` scratch).